
MEET-AI

A modern SaaS platform for hosting video meetings with integrated, context-aware AI agents. MEET-AI combines real-time video conferencing with a generative voice AI that joins the call as a participant, then automatically transcribes, summarizes, and lets you chat with the meeting afterward using full transcript context.

Live Demo: meet-ai-self.vercel.app

Project Documentation Generated June 26, 2026

Table of Contents

- [Problem Statement](#)
 - [Who It's For](#)
 - [Key Features](#)
 - [Tech Stack](#)
 - [Architecture & Data Flow](#)
 - [Project Structure](#)
 - [Setup & Installation](#)
 - [Environment Variables](#)
 - [Running the AI Agent Worker](#)
 - [Development Commands](#)
 - [Security](#)
 - [Deployment](#)
 - [Known Limitations](#)
 - [Roadmap](#)
 - [License](#)
-

Problem Statement

Video conferencing is a cornerstone of modern work, but traditional meetings lack intelligent assistance. Teams routinely struggle with:

- Manual note-taking and meeting documentation
- No contextual AI support *during* the conversation

- The inability to quickly review and discuss what was said afterward
- Meeting insights that must be aggregated by hand

MEET-AI solves this by embedding an intelligent voice agent directly into the video call. The agent listens, transcribes, and speaks in real time, then automatically turns the transcript into a structured summary and exposes a post-meeting chat grounded in that summary.

Who It's For

- **Remote teams** that need AI-powered meeting assistance and automatic documentation
- **Organizations** that require consistent, searchable meeting summaries
- **Builders** creating AI-native communication workflows
- **Distributed teams** across time zones who rely on async meeting context

Key Features

1. Video meetings with a real-time AI voice agent

- Create custom AI agents with their own name and instructions.
- Launch a video call where the agent joins as a live participant.
- The agent listens, responds, and speaks naturally using the **OpenAI Realtime API** (`gpt-realtime`) running inside a **LiveKit Agents** worker.
- Server-side Voice Activity Detection (VAD) and Whisper transcription drive natural turn-taking.

2. Automatic meeting summarization

- When a room closes, summarization runs asynchronously via **Inngest**.
- **GPT-4o** produces a structured Markdown summary containing:
 - A narrative **Overview** of key topics and takeaways
 - **Notes** broken into thematic sections with bullet points
 - Speaker attribution mapped back to real user / agent records
- Heavy LLM work is offloaded to a background queue so it never blocks a webhook response.

3. Post-meeting interactive chat

- Continue the conversation with the agent after the call, in context.
- The agent keeps its original instructions and grounds answers in the generated summary.
- Recent message history is included for coherent, multi-turn follow-ups.
- Powered by **Stream Chat**.

4. Custom AI agent creation

- Define an agent's personality and behavior through free-form instructions.

- Agents are persisted and reusable across many meetings.
- Each meeting carries its agent's instructions into the room via room metadata.

5. Cloud recording & transcripts

- Each room is recorded via **LiveKit Egress** and stored on **Cloudflare R2** (S3-compatible).
- The agent worker captures a structured JSONL transcript and uploads it to R2 for summarization.

6. Authentication & user management

- Email/password plus GitHub and Google OAuth via **Better Auth**.
- Secure, HTTP-only cookie sessions.
- Auto-generated avatars for users and agents.

7. Monetization

- Subscription and checkout flow via **Polar** (sandbox by default, production behind an env flag).

Tech Stack

Frontend

- **Next.js 16** (App Router) — full-stack React framework
- **React 19** — UI library
- **Tailwind CSS 4** — utility-first styling
- **Radix UI / shadcn-style components** — accessible primitives
- **TanStack React Query** — server state management
- **React Hook Form + Zod** — forms and validation
- **LiveKit Components React** — in-call video UI

Backend

- **Next.js App Router** — API routes and server logic
- **tRPC** — end-to-end typed RPC
- **Drizzle ORM** — schema, queries, and migrations
- **Neon** — serverless PostgreSQL
- **Better Auth** — authentication and OAuth

Real-Time & AI

- **LiveKit Cloud** — WebRTC rooms, server SDK, Egress recording
- **LiveKit Agents** — the AI participant worker
- **OpenAI Realtime API** (`gpt-realtime`) — in-call voice agent
- **OpenAI GPT-4o** — summarization and post-meeting chat
- **Stream Chat** — post-meeting messaging

Infrastructure & Services

- **Inngest** — background job queue for async summarization
- **Cloudflare R2** — object storage for recordings and transcripts
- **Upstash Redis** — distributed rate limiting
- **Polar** — subscriptions and payments
- **Vercel** — hosting and deployment

Tooling

- **TypeScript**, **ESLint**, **Drizzle Kit**, **tsx**, **patch-package**

Architecture & Data Flow

MEET-AI is event-driven. The browser talks to the Next.js app over **trRPC** for typed data, connects to a **LiveKit** room for media, and uses **Stream Chat** for post-meeting messaging. LiveKit and Stream both call back into the app via signed webhooks, which fan heavy work out to **Inngest**.

Real-time call flow

1. A user creates a meeting tied to a specific agent. The server creates a LiveKit room (named after the meeting id) and stores the agent's instructions in **room metadata**.
2. The browser requests a join token from `/api/livekit-token`. The route authorizes the caller against the meeting (owner-only today) before minting the token.
3. The user joins the room. A **LiveKit Agents** worker picks up the dispatch, reads the room metadata, and joins as the AI participant.
4. The agent connects the **OpenAI Realtime** model (`gpt-realtime`, `shimmer` voice, server VAD, Whisper transcription) and converses live.
5. As the conversation flows, the worker captures a JSONL transcript, attributing each line to the correct user or agent id.

Recording & end-of-meeting flow

1. On the first human `participant_joined` webhook, the app marks the meeting **active** and starts a **LiveKit Egress** room-composite recording to Cloudflare R2.
2. When the room empties past its `emptyTimeout`, LiveKit fires `room_finished`. The app marks the meeting **processing** and, if a transcript exists, dispatches a `meetings/processing` event to Inngest.
3. When `egress_ended` arrives, the recording's public R2 URL is saved on the meeting.

The meeting ends on `room_finished` (the room emptying), **not** on `participant_left`, so one of several humans leaving never ends the meeting for everyone — this keeps the design multi-user-safe.

Asynchronous summarization flow

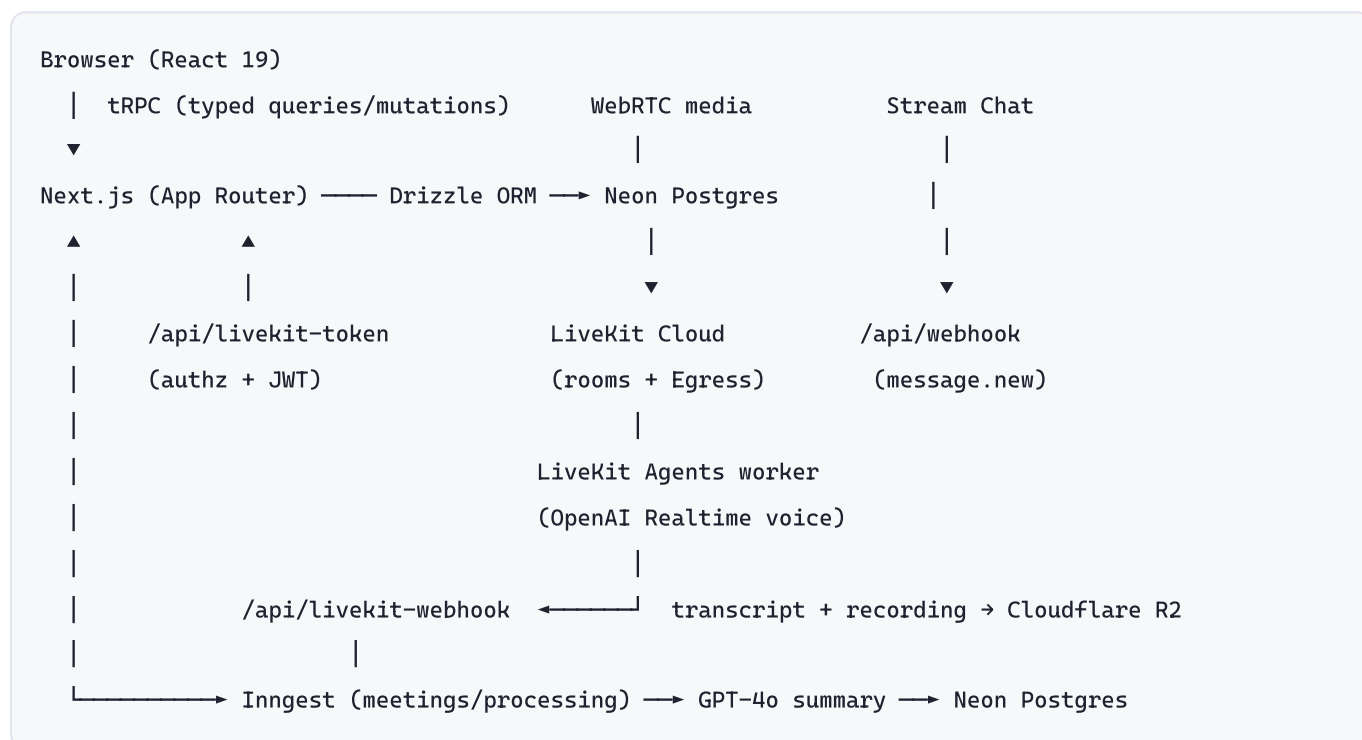
1. The Inngest function fetches the JSONL transcript from R2 (with a hard timeout and a streamed size cap).

2. Speaker ids are mapped to user / agent records for attribution.
3. GPT-4o (via `@inngest/agent-kit`) writes the structured Markdown summary.
4. The summary is saved and the meeting is marked **completed**.

Post-meeting chat flow

1. A chat message fires Stream Chat's `message.new` webhook to `/api/webhook`.
2. The handler verifies the signature, loads the meeting summary and the agent's instructions, and includes recent message history.
3. GPT-4o composes a context-aware reply, which is posted back into the Stream Chat channel as the agent.

Diagram



Project Structure

```
src/
├─ agents/
│   └─ meeting-agent.ts           # LiveKit Agents worker (OpenAI Realtime voice + transcript
capture)
├─ app/
│   ├── (auth)/                   # Sign-in / sign-up routes
│   ├── (dashboard)/              # Agents, meetings, upgrade, home
│   ├── call/[meetingId]/         # In-call experience
│   └─ api/
│       ├── livekit-token/route.ts # Authorized LiveKit join-token minting (SEC-1)
│       ├── livekit-webhook/route.ts # participant_joined / room_finished / egress_ended
│       ├── webhook/route.ts       # Stream Chat message.new (post-meeting chat)
│       ├── ingest/route.ts        # Inngest background worker endpoint
│       └─ trpc/[trpc]/route.ts    # tRPC handler
├─ components/
│   └─ ui/                         # Radix / shadcn-style components
├─ db/
│   ├── index.ts                  # Neon + Drizzle client
│   └─ schema.ts                  # user, session, account, agents, meetings
├─ ingest/
│   ├── client.ts                 # Inngest client
│   └─ function.ts                # Transcript summarization function
├─ lib/
│   ├── auth.ts                   # Better Auth configuration
│   ├── livekit.ts                # LiveKit room/egress clients + token minting
│   ├── stream-chat.ts            # Stream Chat client (post-meeting chat)
│   ├── fetch-transcript.ts       # Bounded, timeout-guarded transcript fetch (SEC-4)
│   ├── ratelimit.ts              # Upstash sliding-window rate limiter (SEC-4)
│   └─ polar.ts                   # Polar payment client (env-driven server)
├─ modules/
│   ├── agents/                   # AI agent CRUD (UI + tRPC procedures)
│   ├── meetings/                 # Meeting history, viewing, creation
│   ├── call/                     # Active call interface (LiveKit room UI)
│   └─ premium/                   # Subscription management
├─ trpc/                          # tRPC routers and setup
└─ constants.ts                   # Shared enums and pagination constants
```

Setup & Installation

Prerequisites

- **Node.js 20+** and npm
- A **Neon** PostgreSQL database
- A **LiveKit Cloud** project (API key, secret, and URL)
- An **OpenAI** API key (Realtime + GPT-4o access)
- A **Stream** account (Chat — used for post-meeting chat)
- A **Cloudflare R2** bucket (recordings + transcripts)
- *(Optional)* GitHub / Google OAuth apps
- *(Optional)* Inngest, Polar, and Upstash Redis accounts

1. Clone the repository

```
git clone https://github.com/theabduallahnadeem/MEET-AI.git
cd MEET-AI
```

2. Install dependencies

```
npm install
```

3. Configure environment variables

Create a `.env.local` file in the project root and fill in the values from the [Environment Variables](#) section.

4. Set up the database

Push the Drizzle schema to your database:

```
npm run db:push
```

Optionally open the visual database studio:

```
npm run db:studio
```

5. Start the development server

```
npm run dev
```

The app runs at `http://localhost:3000`.

6. Start the AI agent worker

In a separate terminal (see [Running the AI Agent Worker](#)):

```
npm run dev:agent
```

Environment Variables

All variables below are set in `.env.local` for local development (and in your hosting provider's dashboard for production).

Application

```
NODE_ENV=development
NEXT_PUBLIC_APP_URL=http://localhost:3000
BETTER_AUTH_SECRET=your_random_secret
```

Database (Neon)

```
DATABASE_URL=postgresql://user:password@host/dbname
```

Authentication (OAuth — optional)

```
GITHUB_CLIENT_ID=your_github_oauth_app_id
GITHUB_CLIENT_SECRET=your_github_oauth_app_secret
GOOGLE_CLIENT_ID=your_google_oauth_app_id
GOOGLE_CLIENT_SECRET=your_google_oauth_app_secret
```

Video & Voice (LiveKit)

```
LIVEKIT_URL=wss://your-project.livekit.cloud
NEXT_PUBLIC_LIVEKIT_URL=wss://your-project.livekit.cloud
LIVEKIT_API_KEY=your_livekit_api_key
LIVEKIT_API_SECRET=your_livekit_api_secret
```

AI (OpenAI)

```
OPENAI_API_KEY=your_openai_api_key
```


Post-meeting Chat (Stream)

```
NEXT_PUBLIC_STREAM_CHAT_API_KEY=your_stream_chat_api_key
STREAM_CHAT_SECRET=your_stream_chat_secret
# Used by the Stream webhook signature verification:
NEXT_PUBLIC_STREAM_API_KEY=your_stream_api_key
STREAM_VIDEO_SECRET=your_stream_video_secret
```

Storage (Cloudflare R2 — S3-compatible)

```
R2_ENDPOINT=https://<account_id>.r2.cloudflarestorage.com
R2_ACCESS_KEY_ID=your_r2_access_key_id
R2_SECRET_ACCESS_KEY=your_r2_secret_access_key
R2_BUCKET=your_bucket_name
R2_PUBLIC_URL=https://your-public-r2-domain
```

Background Jobs (Inngest)

```
INNGEST_SIGNING_KEY=your_inngest_signing_key
INNGEST_EVENT_KEY=your_inngest_event_key
```

Rate Limiting (Upstash Redis — optional, fail-open)

```
UPSTASH_REDIS_REST_URL=your_upstash_redis_rest_url
UPSTASH_REDIS_REST_TOKEN=your_upstash_redis_rest_token
```

Rate limiting is **optional and fails open**: if these are unset, every request passes. Set them in production to activate per-user/per-IP limits on the token and webhook routes.

Monetization (Polar — optional)

```
POLAR_ACCESS_TOKEN=your_polar_access_token
# Defaults to the Polar sandbox. Set to "production" to use live billing:
POLAR_SERVER=sandbox
```

Running the AI Agent Worker

The voice agent runs as a **separate long-lived LiveKit Agents worker** (it is not part of the Next.js server). It connects to your LiveKit project, waits for room dispatches, and joins meetings as the AI participant.

```
# Development (hot-reloading worker)
npm run dev:agent

# Production worker
npm run start:agent
```

The worker (`src/agents/meeting-agent.ts`) reads agent instructions from each room's metadata, runs the OpenAI Realtime model, captures the transcript, and uploads it to Cloudflare R2 on shutdown. For local webhook delivery from LiveKit/Stream, you can expose your dev server with a tunnel:

```
npm run dev:webhook # ngrok tunnel to localhost:3000
```

Development Commands

```
npm run dev          # Start the Next.js dev server
npm run build         # Production build
npm start            # Start the production server
npm run lint          # Run ESLint
npm run db:push       # Apply the Drizzle schema to the database
npm run db:studio     # Open the Drizzle database studio
npm run dev:agent     # Run the LiveKit AI agent worker (dev)
npm run start:agent   # Run the LiveKit AI agent worker (prod)
npm run dev:webhook   # Tunnel local webhooks via ngrok
```

Security

Security is treated as a first-class concern. Recent hardening work includes:

- **Authorized token minting (SEC-1):** `/api/livekit-token` authorizes the caller against the meeting before issuing a LiveKit JWT — deny-by-default, currently owner-only. Designed to widen cleanly to membership-based access for multi-user meetings.
- **Signed webhooks:** Both the LiveKit webhook and the Stream Chat webhook verify request signatures and reject unsigned or tampered payloads.
- **Rate limiting (SEC-4):** Per-user / per-IP sliding-window limits on the public token and webhook routes via Upstash Redis. Fails open so a Redis outage never takes the app down.
- **Bounded transcript fetches (SEC-4):** Server-side transcript downloads enforce a hard timeout and a streamed 5 MB size cap, so a hostile or broken URL can't hang the worker or exhaust memory.
- **Query-injection hardening (SEC-7):** User-supplied search input is escaped for SQL `LIKE` metacharacters before being used in filters.

- **Safe payment defaults (SEC-7):** The Polar client defaults to the sandbox server and only targets production when explicitly configured.

Deployment

The app deploys cleanly to **Vercel**:

1. Push the repository to GitHub.
2. Import the project into Vercel.
3. Set all required environment variables in the Vercel dashboard.
4. Vercel builds and deploys on every push.

The **LiveKit agent worker** is a separate long-running process and should be deployed outside Vercel's serverless model — for example on a container host, VM, or any Node-capable runtime — using `npm run start:agent`.

Known Limitations

- **Transcript attribution:** Summarization relies on speaker ids resolving to user/agent records. If a speaker id can't be matched, that line is attributed to "Unknown."
- **Asynchronous summaries:** Summaries are produced in the background after a room closes; the UI does not stream progress, so a refresh may be needed to see a newly completed summary.
- **Separate worker:** The voice agent requires its own always-on worker process; it does not run inside the Next.js deployment.

Roadmap

Feature	Description
Multi-user knock-to-join	Google Meet–style join requests so non-owners can be admitted to a meeting (extends the deny-by-default token authorization).
Personal AI memory	A persistent per-user assistant that remembers schedules, prior meetings, and context across calls.
Role-aware agents	The agent understands participant roles (PM, Developer, Designer) and tailors its responses.
Real-time fact-checking	The in-call agent queries wikis, issue trackers, or past transcripts to verify claims live.
Sentiment & engagement analytics	A post-meeting dashboard for tone, engagement, and talk-time distribution.
Action-item automation	Automatic extraction and follow-up of action items via Slack / email.
Cross-language translation	Real-time translation so participants can speak their native language.
Meeting search	Full-text / embeddings search across summaries and transcripts.

License

This project is licensed under the MIT License. See the `LICENSE` file for details.

Built with Next.js 16 · React 19 · TypeScript · Tailwind CSS · tRPC · Drizzle · Neon · LiveKit · OpenAI · Stream Chat · Inngest · Cloudflare R2 · Polar